



efreilogo.png

Rapport — Optimisation Convexe

Sujet A : Comparaison des optimiseurs sur CIFAR-10
SGD, SGD+Momentum, Adam, RMSProp, AdaGrad

[Thi Tho PHAM] [Prénom NOM]
[Prénom NOM] [Prénom NOM]

EFREI Paris — Cours d'Optimisation Convexe

Juin 2026

Résumé

Ce rapport présente une étude comparative des principaux algorithmes d'optimisation utilisés pour l'entraînement de réseaux de neurones convolutifs sur le jeu de données CIFAR-10. Nous analysons le comportement de cinq optimiseurs — SGD, SGD avec momentum, Adam, RMSProp et AdaGrad — selon plusieurs critères : vitesse de convergence, précision finale, généralisation et robustesse aux hyperparamètres. Une attention particulière est portée à la nature non convexe du paysage de perte des réseaux profonds, et à la façon dont chaque optimiseur y répond.

Table des matières

1	Introduction	3
2	Fondementals des CNNs et des optimiseurs	3
2.1	Réseaux de neurones convolutifs et apprentissage profond	3
2.2	Descente de gradient stochastique (SGD)	3
2.3	SGD avec momentum	3
2.4	AdaGrad	4
2.5	RMSProp	4
2.6	Adam	4
2.7	Non-convexité et implications pratiques	4
3	État de l'art sur CIFAR-10	5
3.1	Résultats de référence	5
3.2	Comparaisons d'optimiseurs dans la littérature	5
4	Protocole expérimental	5
4.1	Architecture	5
4.2	Données et prétraitement	5
4.3	Optimiseurs et hyperparamètres	5
4.4	Métriques et reproductibilité	6
5	Résultats et analyse	6
5.1	Courbes de convergence	6
5.2	Précision finale et vitesse de convergence	7
5.3	Sensibilité au taux d'apprentissage	7
5.4	Normes des gradients	7
6	Discussion	8
6.1	Lien avec la non-convexité	8
6.2	Adam vs SGD : le paradoxe de la généralisation	8
6.3	Limites de l'étude	9
7	Conclusion	9

1 Introduction

Dans ce projet, nous choisissons d'utiliser un réseau de neurones convolutif (CNN) pour résoudre cette tâche de classification. Les CNNs se sont imposés comme l'architecture de référence pour la vision par ordinateur grâce à leur capacité à extraire automatiquement des caractéristiques visuelles hiérarchiques — des contours et textures aux formes complexes — directement depuis les pixels bruts. Notre objectif n'est pas de maximiser la précision du modèle, mais d'utiliser ce cadre pour observer et comparer le comportement de cinq optimiseurs : SGD, SGD avec momentum, AdaGrad, RMSProp et Adam.

2 Fondementals des CNNs et des optimiseurs

2.1 Réseaux de neurones convolutifs et apprentissage profond

L'apprentissage profond (*deep learning*) s'agit l'entraînement de réseaux de neurones à plusieurs couches, capables d'apprendre automatiquement des représentations hiérarchiques à partir des données brutes. Les CNNs est un l'architecture de référence pour la vision par ordinateur.

Un CNN est composé de trois types de couches : les couches de convolution qui extraient des motifs visuels locaux, les couches de pooling qui réduisent la dimension spatiale, et les couches fully-connected qui produisent la décision finale de classification.

L'entraînement des réseaux revient à minimiser une fonction de coût $\mathcal{L}(\theta)$ non convexe. Nous comparons cinq algorithmes : SGD, SGD avec momentum, AdaGrad, RMSProp et Adam

2.2 Descente de gradient stochastique (SGD)

La descente de gradient stochastique, pour chaque itération, les paramètres θ sont mis à jour dans la direction opposée au gradient de la fonction de coût :

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t) \quad (1)$$

où $\eta > 0$ est le taux d'apprentissage et $\nabla_{\theta} \mathcal{L}$ est le gradient estimé sur un mini-batch. La convergence peut être lente lorsque la surface de perte présente des courbures très différentes selon les directions.

2.3 SGD avec momentum

Le SGD avec momentum utilise un vecteur de vitesse pour la mise à jour de base, ce qui permet de conserver une direction de descente cohérente d'une itération à l'autre :

$$v_{t+1} = \mu v_t + \eta \nabla_{\theta} \mathcal{L}(\theta_t) \quad (2)$$

$$\theta_{t+1} = \theta_t - v_{t+1} \quad (3)$$

Le coefficient $\mu \in [0, 1)$ (typiquement 0,9) contrôle la contribution des gradients passés. Cette mémoire accélère la convergence dans les directions de gradient stable.

2.4 AdaGrad

AdaGrad [6] introduit l'idée d'adapter automatiquement le taux d'apprentissage à chaque paramètre, en le divisant par la racine de la somme cumulée des carrés des gradients passés :

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii} + \epsilon}} g_{t,i} \quad (4)$$

Les paramètres qui reçoivent de grands gradients voient leur taux d'apprentissage diminuer rapidement, tandis que les paramètres peu mis à jour conservent un taux élevé.

2.5 RMSProp

RMSProp [7] reprend l'idée d'AdaGrad mais remplace la somme cumulative par une moyenne mobile exponentielle, ce qui évite que le taux d'apprentissage ne s'annule avec le temps :

$$v_t = \rho v_{t-1} + (1 - \rho) g_t^2 \quad (5)$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t + \epsilon}} g_t \quad (6)$$

avec $\rho \approx 0,9$. Seuls les gradients récents influencent la mise à l'échelle, ce qui rend RMSProp plus stable et efficace pour l'entraînement de réseaux profonds.

2.6 Adam

Adam (Adaptive Moment Estimation) [5] combine les idées du momentum (premier moment) et de l'adaptivité de RMSProp (second moment) :

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (7)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (8)$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t + \epsilon}} m_t \quad (9)$$

avec $\beta_1 = 0,9$, $\beta_2 = 0,999$ et $\epsilon = 10^{-8}$ par défaut. Adam ajoute une correction de biais sur m_t et v_t pour les premières itérations. Sa robustesse au choix du taux d'apprentissage en fait un optimiseur pratique et largement adopté.

2.7 Non-convexité et implications pratiques

Les fonctions de coût des réseaux profonds sont non convexes : elles présentent des minima locaux, des points-selles et des plateaux qui compliquent l'optimisation. En pratique, les optimiseurs basés sur le gradient ne garantissent pas de trouver le minimum global, mais convergent vers des solutions acceptables grâce au bruit stochastique et à la haute dimensionnalité de l'espace des paramètres. Le choix de l'optimiseur influence non seulement la vitesse de convergence, mais aussi la qualité des minima trouvés et donc la capacité du modèle à généraliser.

3 État de l’art sur CIFAR-10

3.1 Résultats de référence

Le tableau 1 résume les performances publiées sur CIFAR-10 pour des architectures comparables.

TABLE 1 – Précision sur CIFAR-10 — état de l’art (sélection)

Modèle	Optimiseur	Précision (%)	Référence
ResNet-20	SGD + momentum	91.2	[2]
VGG-16	SGD + momentum	93.6	[3]
ResNet-56	SGD + momentum	93.0	[2]
DenseNet-100	SGD + momentum	95.5	[4]

3.2 Comparaisons d’optimiseurs dans la littérature

Wilson et al. [8] montrent empiriquement que les méthodes adaptatives (Adam, RMSProp) convergent plus vite mais généralisent moins bien que SGD avec momentum sur les tâches de vision. Ils attribuent cet écart à la géométrie des minima trouvés. Cette observation constitue un point central de notre analyse. Recander de toujours termine par utilise Adam a la fin chaque model

4 Protocole expérimental

4.1 Architecture

Nous utilisons un CNN simple composé de trois blocs convolutifs suivis de deux couches fully-connected. Chaque bloc convolutif contient une convolution, une activation ReLU et une couche de max-pooling. Les canaux passent successivement de 3 à 32, puis 64 et 128. Après les trois blocs, les cartes de caractéristiques sont aplaties, puis envoyées dans une couche linéaire de 256 neurones avant la couche finale de classification à 10 sorties, correspondant aux 10 classes de CIFAR-10.

4.2 Données et prétraitement

CIFAR-10 contient 50 000 images d’entraînement et 10 000 images de test, réparties équitablement sur 10 classes (avion, automobile, oiseau, chat, cerf, chien, grenouille, cheval, bateau, camion). Les augmentations appliquées sont : retournement horizontal aléatoire, recadrage aléatoire (padding=4), normalisation par la moyenne et l’écart-type du jeu d’entraînement.

4.3 Optimiseurs et hyperparamètres

Dans cette partie, nous sommes intéressés au taux d’apprentissage. Donc nous avons utilisé le Grid Search pour chercher le meilleur taux d’apprentissage pour chaque optimiseur et fixer les autres hyperparamètres par les valeurs défaut, avec un but - l’égalité sur le point de départ de l’entraînement model.

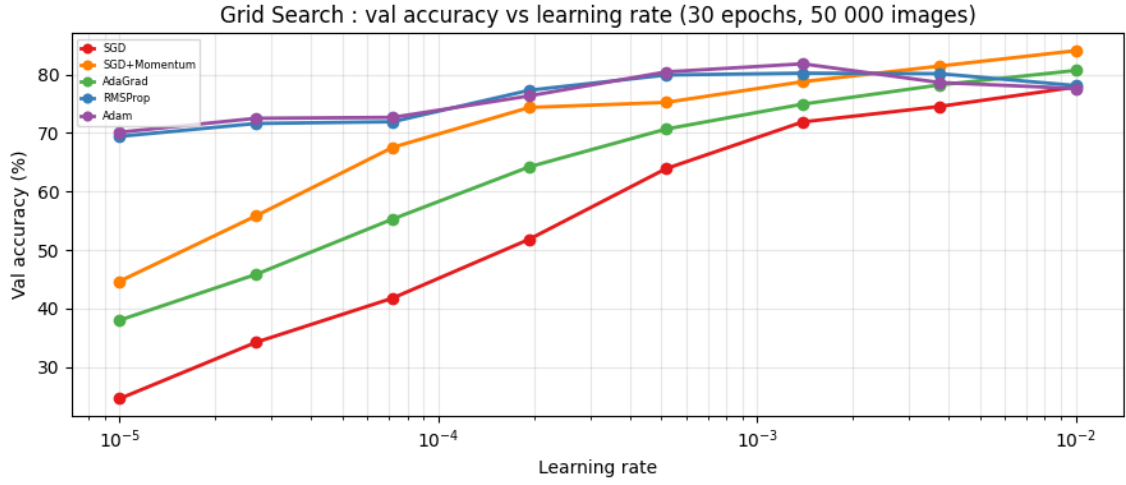


FIGURE 1 – Grid search sur learning rate

TABLE 2 – Résultat le meilleur taux d'apprentissage

Optimiseur	Paramètres	Taux d'apprentissage
SGD	-	10^{-2}
SGD + Momentum	$\mu = 0.9$	10^{-2}
AdaGrad	$\epsilon = 10^{-8}$	10^{-2}
RMSProp	$\rho = 0.9, \epsilon = 10^{-8}$	1.39×10^{-3}
Adam	$\beta_1 = 0.9, \beta_2 = 0.999$	1.39×10^{-3}

4.4 Métriques et reproductibilité

Toutes les expériences sont reproduites avec 42 graines aléatoires différentes. Nous mesurons la précision sur le jeu de validation (10% des données d'entraînement) et sur le jeu de test, ainsi que l'écart de généralisation (précision entraînement – précision test).

5 Résultats et analyse

5.1 Courbes de convergence

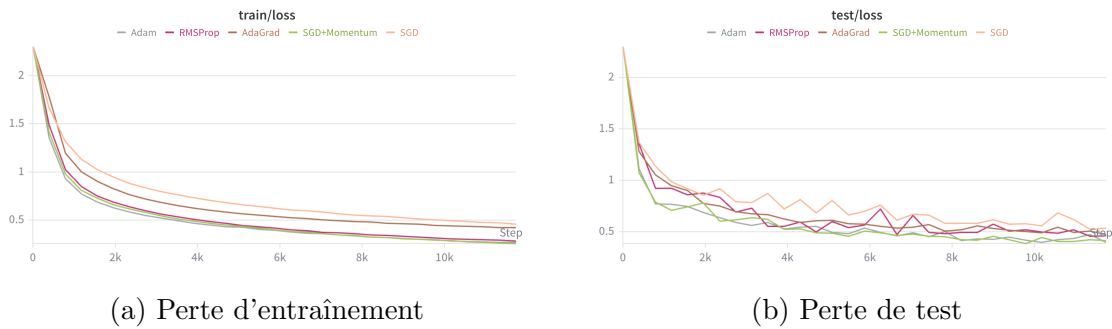


FIGURE 2 – Courbes de perte pour les cinq optimiseurs

Les courbes de perte montrent que les optimiseurs adaptatifs et les méthodes avec momentum convergent plus rapidement que SGD. Adam et SGD+Momentum obtiennent dès les premières époques une perte d'entraînement plus faible, ce qui indique une progression plus rapide dans le paysage de perte. RMSProp présente également une forte amélioration, en particulier sur la perte de test, qui diminue rapidement jusqu'à devenir l'une des plus faibles.

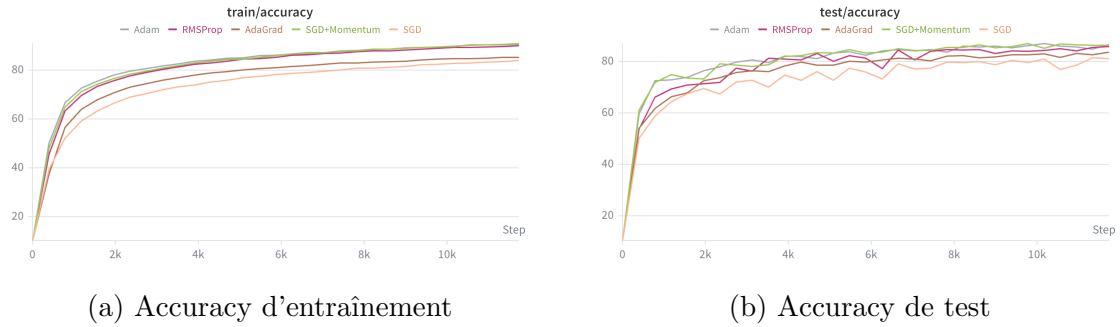


FIGURE 3 – Courbes d'accuracy pour les cinq optimiseurs

À l'inverse, SGD descend plus lentement. Sa perte d'entraînement reste plus élevée que celle des autres optimiseurs sur toute la période observée. Ce comportement est attendu, car SGD utilise un taux d'apprentissage fixe et ne bénéficie ni d'une mémoire de direction comme le momentum, ni d'une adaptation par paramètre comme Adam, RMSProp ou AdaGrad.

5.2 Précision finale et vitesse de convergence

Les résultats montrent qu'Adam et SGD+Momentum sont les optimiseurs les plus performants :

- Adam atteint la meilleure précision test finale (86.74 %), tandis que SGD+Momentum obtient la meilleure précision test maximale (87.05 %), un écart négligeable qui en fait deux choix équivalents dans notre configuration. RMSProp se situe juste en dessous (85.84 %), suivi d'AdaGrad (83.77 %), dont le faible écart de généralisation ne compense pas une précision finale plus modeste. SGD seul atteint 81.22 %, progressant correctement mais trop lentement sur 30 epochs sans mécanisme d'adaptation.

Ces résultats illustrent un compromis entre vitesse de convergence et généralisation : les méthodes adaptatives et à momentum atteignent de meilleures précisions mais présentent un écart train-test plus marqué. Dans un budget de 30 epochs, Adam et SGD+Momentum s'imposent comme les choix les plus compétitifs.

5.3 Sensibilité au taux d'apprentissage

5.4 Normes des gradients

[Insérer figure de l'évolution de la norme $\|\nabla_{\theta}\mathcal{L}\|$ au cours de l'entraînement]

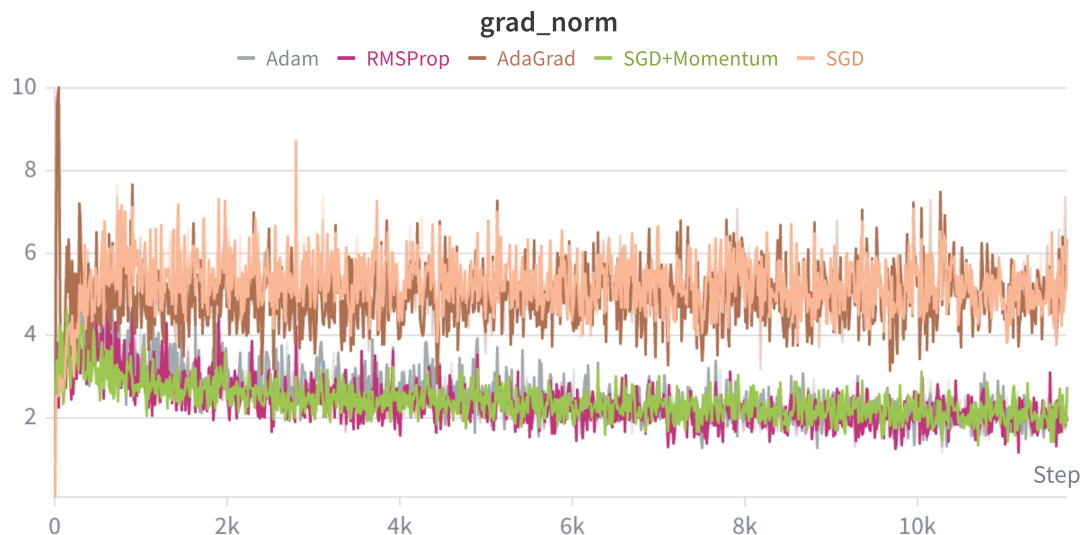


FIGURE 4 – Normes des gradients

SGD+Momentum, Adam et RMSProp montrent une diminution plus nette de la norme des gradients. Donc ces optimiseurs stabilisent progressivement l’entraînement et se rapprochent d’une zone où les mises à jour deviennent plus petites. SGD+Momentum présente une norme particulièrement faible en fin d’entraînement, ce qui reflète une trajectoire d’optimisation plus régulière.

AdaGrad conserve une norme de gradient assez élevée mais relativement stable. Son comportement est différent des autres méthodes adaptatives, car son taux d’apprentissage effectif diminue continuellement avec l’accumulation des gradients. Cela peut expliquer sa progression correcte mais moins compétitive sur la précision finale.

6 Discussion

6.1 Lien avec la non-convexité

Les résultats illustrent les difficultés de l’optimisation non convexe : les optimiseurs suivent des trajectoires différentes, visibles dans les courbes de perte, les normes de gradient et les écarts de généralisation. SGD présente une convergence plus lente et des gradients plus bruités, tandis qu’Adam, RMSProp et SGD+Momentum stabilisent plus rapidement l’apprentissage grâce au momentum et à l’adaptation du taux. Cependant, une convergence rapide s’accompagne d’un écart train-test plus important, suggérant que la trajectoire influence non seulement la vitesse, mais aussi le type de minimum atteint.

6.2 Adam vs SGD : le paradoxe de la généralisation

Les résultats confirment partiellement [8] : Adam obtient la meilleure précision test finale (86.74 %), mais SGD+Momentum atteint la meilleure précision maximale (87.05 %), pour un écart de généralisation de 4.55 points contre 3.93 pour Adam. Dans notre configuration, Adam ne généralise donc pas moins bien que SGD ; le paradoxe se manifeste plutôt comme un compromis entre rapidité de convergence, précision finale et écart train-test.

6.3 Limites de l'étude

Le budget fixe de 30 epochs favorise les optimiseurs rapides ; un entraînement plus long pourrait avantager SGD. Les résultats issus d'une seule exécution par optimiseur ne permettent pas d'estimer la variance due à l'initialisation ou au bruit stochastique. Enfin, la comparaison reste sensible aux hyperparamètres choisis et à l'architecture utilisée ; les conclusions sont donc propres à ce cadre expérimental.

7 Conclusion

Ce travail a mis en évidence les différences de comportement des optimiseurs classiques sur un problème de classification non convexe. [Résumer les 2-3 conclusions principales]. Ces résultats rappellent que le choix de l'optimiseur ne se réduit pas à sa vitesse de convergence : la géométrie du paysage de perte et la capacité à généraliser sont des critères tout aussi déterminants.

Références

- [1] Krizhevsky, A. (2009). *Learning Multiple Layers of Features from Tiny Images*. Technical Report, University of Toronto.
- [2] He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition*. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 770–778.
- [3] Simonyan, K., & Zisserman, A. (2014). *Very Deep Convolutional Networks for Large-Scale Image Recognition*. International Conference on Learning Representations (ICLR).
- [4] Huang, G., Liu, Z., Van der Maaten, L., & Weinberger, K. Q. (2017). *Densely Connected Convolutional Networks*. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 4700–4708.
- [5] Kingma, D. P., & Ba, J. (2015). *Adam : A Method for Stochastic Optimization*. International Conference on Learning Representations (ICLR).
- [6] Duchi, J., Hazan, E., & Singer, Y. (2011). *Adaptive Subgradient Methods for Online Learning and Stochastic Optimization*. Journal of Machine Learning Research, 12, 2121–2159.
- [7] Tieleman, T., & Hinton, G. (2012). *Lecture 6.5 — RMSProp : Divide the gradient by a running average of its recent magnitude*. COURSERA : Neural Networks for Machine Learning.
- [8] Wilson, A. C., Roelofs, R., Stern, M., Srebro, N., & Recht, B. (2017). *The Marginal Value of Momentum for Small Learning Rate SGD*. International Conference on Learning Representations (ICLR).